



UNIVERSIDADE DO ESTADO DO AMAZONAS
ESCOLA SUPERIOR DE TECNOLOGIA
NÚCLEO DE COMPUTAÇÃO - NUCOMP

ALGORITMOS E ESTRUTURAS DE DADOS II 2026/2 (versão 3)
PROF. FLÁVIO JOSÉ MENDES COELHO

PROJETO PRÁTICO 2

1 Objetivos

Este **Projeto Prático 2 – PP2**, tem o objetivo de exercitar e avaliar suas habilidades em:

- Aplicar os algoritmos e estruturas de dados abordados em AED 2 em um problema contextualizado;
- Aplicar e codificar o tipo abstrato de dados grafo na linguagem de programação exigida neste enunciado, aplicando **busca em largura** para solucionar diversos subproblemas relacionados ao problema principal;
- Avaliar seu domínio sobre o código que você próprio desenvolveu neste projeto, a partir dos assuntos abordados na disciplina AED 2.

2 Descrição do problema

ATAQUE DOS CAVALEIROS

Em um tabuleiro de xadrez há quatro cavaleiros pretos e um rei branco em posições aleatórias. Seu objetivo é desenvolver um programa para descobrir quais cavaleiros podem ameaçar o rei, realizando o mínimo de movimentos até ele. Uma posição de ameaça ao rei é aquela posição alcançada após zero ou mais movimentos de um cavaleiro, a partir da qual o cavaleiro alcançaria a posição do rei com apenas mais um movimento. Note que um cavaleiro movimenta-se em “L” (consulte um manual de Xadrez).

Observe na Figura 1, no tabuleiro à direita, que:

- O cavaleiro preto na posição **h1** move-se para a posição **g3**, e ameaça o rei.
- O cavaleiro preto na posição **c7** move-se para **b5** e, depois, para **d6**, e ameaça o rei.
- O cavaleiro preto na posição **e8** move-se para a posição **f6**, e ameaça o rei.
- O cavaleiro preto na posição **a1** move-se para a posição **b3**, em seguida, move-se para a posição **d2**, e ameaça o rei.

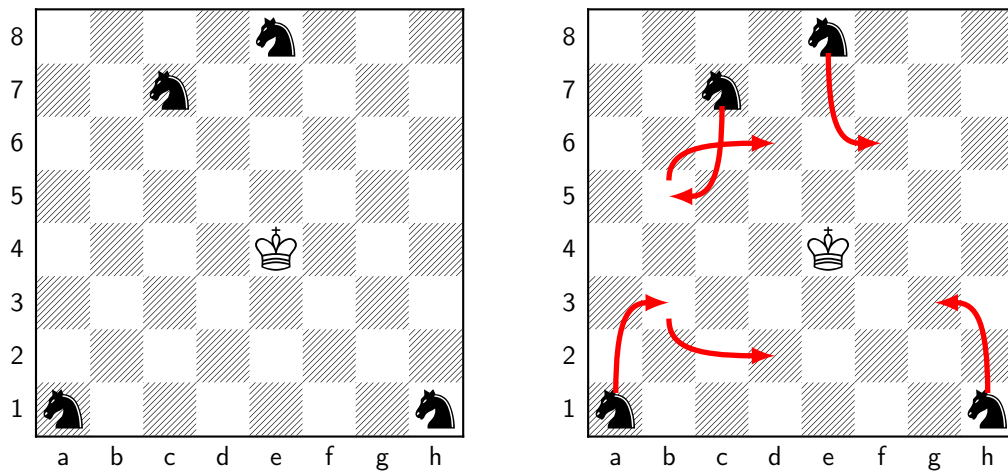


Figura 1: Exemplo 1 de movimentos dos cavaleiros em ameaça ao rei.

No exemplo acima, somente os cavaleiros em h1 e e8 fazem o número mínimo de movimentos até uma posição de ameaça ao rei. Agora, veja a Figura 2.

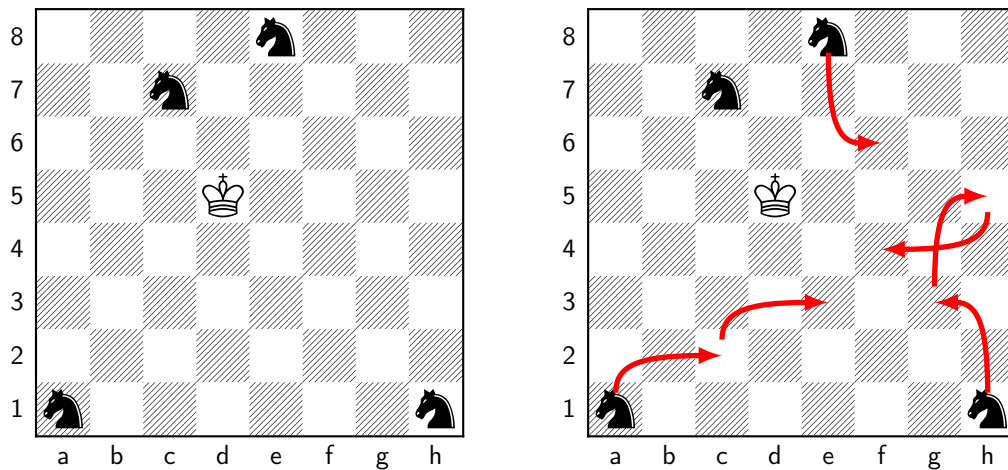


Figura 2: Exemplo 2 de movimentos dos cavaleiros em ameaça ao rei.

Nesse segundo exemplo, observe no tabuleiro à direita, que:

- O cavaleiro em h1 realiza 3 movimentos para ameaçar o rei.
- O cavaleiro em e8 realiza somente 1 movimento para ameaçar o rei.
- O cavaleiro em a1 realiza 2 movimentos para ameaçar o rei.
- O cavaleiro em c7 precisa de 0 movimentos para ameaçar o rei, pois ele já inicia em uma posição de ameaça ao rei.

Sua missão é descobrir quais cavaleiros ameaçam o rei e com quantos movimentos. Na saída, mostre o número mínimo de movimentos. Para isso:

- (1) Codifique um **TAD grafo** com **lista de adjacência** para representar o **grafo de caminhada de um cavaleiro**;
- (2) Codifique a **busca em largura (BFS)** para encontrar a solução para esse problema;
- (3) Envie seu código para o juiz on-line Hackerrank para passar em todos os casos de teste.

3 Entradas e saídas do problema

Entrada. O número de testes a serem executados, seguido das posições dos quatro cavaleiros pretos, e a posição do rei branco, separados por espaço e seguidos de um **enter**. Veja a entrada e a saída para os dois exemplos dados:

```
2
h1 c7 e8 a1 e4
h1 c7 e8 a1 d5
```

Saída. Dentre os cavaleiros, apresente o número de movimentos daquele que revelar ter o número mínimo de movimentos até uma ameaça ao rei. Se houver mais de um cavaleiro com o mesmo número mínimo, apresente esse número para cada um desses cavaleiros, na ordem em que eles aparecem na entrada:

```
1 1
0
```

Na primeira linha dessa saída, o primeiro 1 se refere ao cavaleiro em **h1** e o segundo 1 se refere ao cavaleiro em **e8**. Na segunda linha da saída, 0 se refere ao cavaleiro em **c7**.

4 Requisitos do projeto

1. **Equipes.** Este projeto deve ser desenvolvido individualmente. **Excepcionalmente, poderá ser realizado em dupla, mediante autorização do professor por razão justificável.**
2. **Ferramentas e técnicas.** O projeto deve ser codificado em C++20. Da STL, são permitidos apenas os contêineres `std::list`, `std::vector`, `std::queue` e `std::pair`; qualquer outro contêiner da STL não é permitido. O uso de qualquer outra biblioteca não é permitido, com exceção de `<iostream>`, `<format>`, `<cmath>`, `<string>`, `<string_view>`, `<cctype>`, `<iomanip>`, `<limits>` e `<optional>`. Pelo menos o TAD grafo deve ser codificado com programação orientada a objetos (classe, encapsulamento e *templates* quando aplicável). No restante do código, você pode usar POO ou programação procedural conforme sua escolha.
3. **Submissão.** Todo o projeto deverá ser submetido ao juiz on-line (a ser informado pelo professor) em um único arquivo fonte com extensão `cpp`.

4. **Correção e pontuação.** A correção e pontuação do PP2 é dividida em duas etapas:

Correção funcional – CF. Realizada de forma automática pelo juiz on-line. Pontuação máxima: 10,0 pts. O aluno pode submeter o código quantas vezes quiser até o prazo final. A pontuação corresponde a $10,0/N \times C$, onde N é o número total de casos de teste e C é o número de casos acertados pelo aluno.

Correção de Inspeção de código – IC. Realizada manualmente pelo professor, que verificará se o código atende aos requisitos deste enunciado. Em caso de não conformidade, a pontuação obtida na CF sofrerá penalizações primeiro conforme as condições abaixo.

Condições necessárias para a correção da IC: o não cumprimento de qualquer uma das condições abaixo implica em $CF = 0.0$ e $IC = 0.0$:

- (a) O código do projeto foi submetido ao juiz on-line em um único arquivo .cpp e obteve $CF > 0$;
- (b) Todo o projeto está codificado exclusivamente em **C++20**;
- (c) Todo o projeto **não** utiliza herança, classes *friend*, operator, funções lambda e qualquer outro comando ou recurso avançado do C++20 não tratado nas aulas de AED2. O tratamento de exceções **é permitido**;
- (d) Todo o projeto utiliza **somente** as bibliotecas `<iostream>`, `<format>`, `<cmath>`, `<string>`, `<string_view>`, `<cctype>`, `<iomanip>`, `<limits>`, `<optional>` e os contêineres permitidos no item 2 (`<list>`, `<vector>`, `<queue>`, `<utility>`).

Se **qualquer uma** das condições não for atendida, então a nota do PP2 será $PP2 = 0$. Caso todas as condições sejam atendidas, será verificado se os requisitos da Tabela 1 foram atendidos. Para cada requisito não atendido será acumulada a penalidade correspondente (coluna **Peso na IC**), totalizando um valor IC (total de penalidades). Ao final, a nota do PP2 será calculada por $PP2 = CF - (CF \times IC / 100)$.

Tipo	Requisito	Peso na IC
AED	Codifique o TAD Grafo com lista de adjacência sob a forma de uma classe (class , não struct), conforme orientado no PP1, com a interface pública composta pelos métodos de adição de uma aresta, obtenção da lista de adjacência de um vértice e impressão da lista de adjacência	20%
AED	Construa o grafo de caminhada do cavaleiro : as 64 casas do tabuleiro como vértices, e arestas entre casas alcançáveis por um único movimento em “L” válido no tabuleiro 8×8	20%
AED	Codifique a busca em largura (BFS) conforme a versão das aulas de AED2 (que é a versão do Cormen), como uma função ou como um método de uma classe, à sua escolha. Espera-se que a BFS seja processada sobre o grafo de caminhada do cavaleiro.	20%
AED	Implemente a lógica da solução em uma função própria ou como um método de uma classe própria, fora da função main . Essa lógica corresponde a: (1) obter o número mínimo de movimentos por cavaleiro; (2) obter qual foi o movimento mínimo geral entre os cavaleiros; (3) tratar o caso em que há mais de um movimento mínimo (empate entre cavaleiros). Essa função/método poderá ser quebrada em outras subfunções menores, à sua escolha.	20%
Programação	Faça uso correto de funções e parâmetros (ponteiros, referências e const), constantes, enumerações, <i>templates</i> (quando aplicável), estruturas e classes C++. Nenhuma variável global deve ser utilizada	10%
Estilo de código	Use endentação no padrão K&R ou Allman (veja https://en.wikipedia.org/wiki/Indentation_style) e nomeação de identificadores com <i>Camel case</i> , <i>Snake case</i> ou <i>Pascal case</i> (veja https://en.wikipedia.org/wiki/Naming_convention_(programming))	5%
	Use um mesmo padrão de estilo de nomeação de identificadores (incluindo usar um único idioma em todo o código) e garanta que eles expressem bem os conceitos do problema (veja https://t.ly/2owrY e pule direto para “P.1: Express ideas directly in code”)	5%
Total máximo de penalidades		100%

Tabela 1: Requisitos do PP2 e seus pesos na Inspeção de Código (IC).

Datas

- Emissão deste enunciado: 08/09/2026 às 10:30 (hora local).
- Abertura do juiz online: 11/09/2026 às 09:30 (hora local).
- Fechamento do PP2 no juiz on-line: 21/09/2026 às 22:00 (hora local).

CÓDIGO DE ÉTICA

Este projeto deve ser concebido, projetado, codificado e testado pelo próprio autor (ou equipe, se for o caso), com base nas referências bibliográficas fornecidas neste enunciado e nas aulas de Algoritmos e Estruturas de Dados, ou por outras referências bibliográficas indicadas pelo professor. Portanto, não copie código pronto da internet ou de serviços/aplicativos/IA para aplicá-lo a este projeto, nem copie código de colegas, ou mesmo permita que terceiros produzam este trabalho em seu lugar. Isso prejudica a sua experiência de aprendizado, proposta por esse trabalho, fere o código de ética desta disciplina e implica na atribuição da nota mínima ao trabalho.

Referências

- [1] C++. In: *WIKIPÉDIA, a enciclopédia livre*. Flórida: Wikimedia Foundation, 2016. Disponível em: <<https://pt.wikipedia.org/w/index.php?title=C%2B%2B&oldid=45048480>>. Acesso em: 17 abr. 2016.
- [2] C++. In: *cppreference.com*, 2016. Disponível em <<http://en.cppreference.com/w/>>. Acesso em: 17 abr. 2016.
- [3] CORMEN, T. H., Leiserson, C. E., Rivest, R. L., Stein C. *Introduction to Algorithms*, 3rd edition, MIT Press, 2010
- [4] STROUSTRUP, Bjarne. *The C++ Programming Language*. 4th. Edition, Addison-Wesley, 2013.
- [5] STROUSTRUP, Bjarne. *A Tour of C++*. Addison-Wesley, 2014.
- [6] SEDGEWICK, R., WAYNE, K. *Algorithms*. 4th edition, Addison-Wesley
- [7] SZWARCFITER, Jayme Luiz et. alii. *Estruturas de Dados e seus Algoritmos*. Rio de Janeiro. 2a. Ed. LTC, 1994.
- [8] WIRTH, Niklaus. *Algoritmos e Estruturas de Dados*. Rio de Janeiro. 1a. Ed. Prentice - Hall do Brasil Ltda., 1989.
- [9] ZIVIANI, Nívio. *Projeto de Algoritmos com Implementação em Java e C++*. 2a. Edição. Cengage Learning, 2010.
- [10] ZIVIANI, Nívio. *Projeto de Algoritmos com Implementação em Pascal e C*. 3a. Ed. São Paulo: Cengage Learning, 2012.